

OPTIMIZACIÓN MEDIANTE PRUNING Y COMPRESIÓN LIGERA PARA LA EJECUCIÓN EFICIENTE DE MODELOS DE IA EN DISPOSITIVOS DE BORDE PARA ENTORNOS IOT CON RECURSOS LIMITADOS, 2026.

Carlos Henry Santana Palomino

FIEE, Universidad Nacional de Ingeniería

Inteligencia Artificial II

Yury Oscar Tello Canchapoma

15 de mayo de 2026

RESUMEN

El despliegue de sistemas de detección de intrusiones basados en Inteligencia Artificial en dispositivos de borde (*edge*) para el Internet de las Cosas (IoT) enfrenta cuellos de botella críticos debido a las severas restricciones de memoria, almacenamiento y procesamiento. El presente estudio tuvo como objetivo principal evaluar la influencia de la optimización de modelos mediante *pruning* (selección de características) y compresión ligera en la ejecución eficiente del algoritmo LightGBM. Se aplicó un diseño experimental comparativo utilizando un subconjunto del *Edge-IIoTset*, conformando una línea base de 42 características y comparándola contra estrategias de poda (reteniendo el Top 20 de características), serialización comprimida (Joblib nivel 9) y una co-optimización híbrida. Los resultados demostraron que la estrategia combinada mitiga significativamente las limitaciones de hardware: logró reducir el tamaño de almacenamiento del modelo en un 60.71% (de 7230.03 KB a 2840.76 KB) y disminuyó la latencia de inferencia en un 23.88% (de 117.22 ms a 89.23 ms). Lejos de comprometer la capacidad de detección, la reducción de dimensionalidad estabilizó el modelo, incrementando marginalmente la eficacia predictiva con un *Accuracy* y *Recall* de 0.9267, y un *F1-Score* de 0.9290 (+0.09% respecto al baseline).

Se concluye que la implementación conjunta de *pruning* y compresión ligera es una técnica altamente efectiva para viabilizar el despliegue ágil y seguro de modelos en entornos IoT industriales con recursos limitados.

PALABRAS CLAVE: Internet de las Cosas (IoT), Edge Computing, LightGBM, Pruning, Compresión de Modelos, Detección de Anomalías, Ciberseguridad.

IDEAS DESTACADAS

- **Viabilidad en el borde:** La co-optimización de modelos LightGBM resuelve los cuellos de botella de hardware en arquitecturas IoT sin necesidad de aplicar cuantización compleja.
- **Eficiencia de almacenamiento:** La compresión ligera mediante la serialización con Joblib redujo la huella de memoria del modelo en un 60.71%, ocupando solo 2840.76 KB.
- **Aceleración de inferencia:** El *pruning* agresivo, al retener solo las 20 características más importantes (reducción del 52.4% de la dimensionalidad), agilizó la latencia de inferencia en un 23.88% (89.23 ms).
- **Eficacia preservada:** A pesar de la drástica reducción de recursos, el modelo híbrido no presentó degradación predictiva, logrando una leve mejora del +0.09% en el *F1-Score* (0.9290) frente al modelo original.

1. INTRODUCCIÓN

La integración del Internet de las Cosas (IoT) ha impulsado la interconexión de dispositivos físicos en diversos dominios, tales como infraestructuras industriales, sistemas de salud y ciudades inteligentes, generando flujos masivos de datos en tiempo real (Akhi et al.,

2025; Jan et al., 2019). Para procesar esta información con baja latencia, mejorar la privacidad y evitar la dependencia exclusiva de servidores centralizados, la computación de borde (*edge computing*) ha emergido como un paradigma fundamental que acerca la inteligencia computacional directamente a la fuente de generación de los datos (Fridous et al., 2026; Miranto et al., 2026). Sin embargo, el despliegue de soluciones en estos entornos enfrenta un desafío crítico, ya que los dispositivos IoT de borde operan bajo severas restricciones de capacidad de procesamiento, memoria disponible y presupuesto energético (Hadi et al., 2026; Khowaja et al., 2025).

Estas limitaciones de hardware representan un obstáculo mayor para la implementación de modelos predictivos avanzados, especialmente en tareas críticas de ciberseguridad como la detección de anomalías y ataques de red, las cuales exigen respuestas inmediatas (Adjewa et al., 2026; Roy et al., 2021). Los algoritmos tradicionales de aprendizaje automático y aprendizaje profundo demandan una cantidad de recursos computacionales (RAM, CPU y almacenamiento) que excede con creces las capacidades de los nodos embebidos (Wang et al., 2022). Por consiguiente, el desarrollo y adopción de modelos ligeros de Inteligencia Artificial (IA) se ha vuelto imperativo para garantizar la seguridad perimetral de las redes y sistemas ciberfísicos sin agotar los escasos recursos del dispositivo (Jan et al., 2019; Roy et al., 2021).

Para cerrar la brecha existente entre la alta eficacia predictiva lograda en laboratorios y la viabilidad operativa real del hardware de borde, la literatura reciente destaca el uso imperativo de estrategias de optimización de modelos, tales como el *pruning* (poda) y la compresión ligera (Fridous et al., 2026; Miranto et al., 2026). El *pruning* permite reducir la dimensionalidad y la redundancia estructural de los datos de entrada —por ejemplo, seleccionando y reteniendo únicamente las características más importantes— lo que disminuye drásticamente la carga de procesamiento algorítmico y, por ende, la latencia de

inferencia (Hadi et al., 2026; Khowaja et al., 2025). En paralelo, las técnicas de compresión ligera intervienen directamente en la persistencia del modelo mediante una serialización eficiente, minimizando su huella física de almacenamiento en disco sin alterar su arquitectura interna (Miranto et al., 2026).

A pesar de los beneficios demostrados por estas técnicas, persiste una brecha de despliegue en la aplicación combinada de estos métodos sobre algoritmos basados en árboles de decisión altamente eficientes (como LightGBM) en entornos con arquitecturas limitadas (Hadi et al., 2026). La motivación de este estudio radica en abordar esta deficiencia proporcionando una solución empírica que asegure que los sistemas de detección de intrusiones puedan ejecutarse localmente de manera ágil, mitiguen la dependencia de la nube y reduzcan los riesgos de seguridad en arquitecturas de nueva generación (Roy et al., 2021). En este contexto, el objetivo principal de la presente investigación es evaluar la influencia de la optimización mediante *pruning* y compresión ligera en la ejecución eficiente de modelos de IA en dispositivos de borde para entornos IoT con recursos limitados en 2026.

2. REVISIÓN DE LITERATURA

El despliegue de sistemas de detección de intrusiones (IDS) en redes de Internet de las Cosas (IoT) enfrenta el reto crítico de procesar datos en tiempo real bajo severas restricciones de memoria, energía y capacidad de cómputo. Para mitigar estos cuellos de botella, la comunidad científica ha transitado hacia la computación de borde (edge AI) y el diseño de modelos ligeros (lightweight models).

En el ámbito de la detección de intrusiones para IoT, investigaciones previas han priorizado la reducción de la complejidad del modelo. Jan et al. (2019) propusieron un IDS ligero basado en Máquinas de Vectores de Soporte (SVM) que extrae un conjunto mínimo de características (basado en la tasa de llegada de paquetes), demostrando que la eliminación de

etapas complejas de extracción reduce significativamente el consumo de recursos en nodos sensores. De manera complementaria, Roy et al. (2021) abordaron las limitaciones de los dispositivos IoT implementando un mecanismo de detección de intrusiones que utiliza eliminación de multicolinealidad y reducción de dimensionalidad acoplado a un clasificador B-Stacking, logrando disminuir drásticamente el tamaño de los datos de entrenamiento y el tiempo de respuesta.

A medida que las arquitecturas de edge AI evolucionan, la aplicación de técnicas de optimización estructural, como la poda (pruning) y la selección de características, se ha vuelto indispensable para reducir el peso de almacenamiento y la latencia. En este contexto, Khan et al. (2025) desarrollaron un enfoque híbrido para la detección de intrusiones empleando un algoritmo genético para la selección agresiva de características. Combinado con una cuantización dinámica post-entrenamiento, su modelo de aprendizaje profundo (DNN-BiLSTM) logró comprimirse a solo 108.42 KB, manteniendo una precisión del 99.84% y probando su viabilidad en hardware IoT restringido. Asimismo, en un análisis exhaustivo sobre hardware heterogéneo, Fridous et al. (2026) demostraron a través de un marco de benchmarking que la poda y la compresión reducen sustancialmente la huella de memoria y el consumo energético de las redes neuronales en microcontroladores y plataformas de borde.

Enfocándose en modelos basados en árboles de decisión por su eficiencia inherente, Hadi et al. (2026) realizaron un análisis comparativo de estrategias de poda y cuantización específicamente para el algoritmo LightGBM desplegado en dispositivos edge (Raspberry Pi 4). Su estudio comprobó que el pruning basado en la regularización de hiperparámetros disminuye de forma extrema el tamaño del modelo (hasta un 97.3%) sin afectar negativamente la capacidad de detección de anomalías. Adicionalmente, Khowaja et al. (2025) introdujeron un marco de seguridad orientado al pruning (POST) para sistemas IoT de

borde, evidenciando que la poda estructurada no solo optimiza las cargas de comunicación y computación, sino que incrementa la seguridad del modelo frente a ataques de inversión, al limitar la cantidad de información redundante expuesta.

Relación con la presente investigación Los antecedentes expuestos confirman que la combinación de selección de características (pruning) y compresión de modelos es el estándar óptimo para llevar la inteligencia artificial al borde de la red. La presente investigación se alinea directamente con los hallazgos de Hadi et al. (2026) y Khan et al. (2025), adoptando el eficiente algoritmo LightGBM para la detección de intrusiones. Sin embargo, a diferencia de los enfoques que emplean cuantización compleja a enteros (INT8) o algoritmos genéticos iterativos que requieren alto poder de cómputo para su optimización previa, este estudio propone una co-optimización simplificada y altamente pragmática. A través de un *pruning* selectivo basado estrictamente en el ranking de importancia de características (reteniendo el Top 20) y una serialización mediante compresión ligera con *Joblib*, esta investigación demuestra que es posible lograr reducciones del tamaño de almacenamiento superiores al 60% y disminuciones significativas de latencia sin alterar la arquitectura interna matemática, ofreciendo una solución de despliegue ágil y altamente compatible con las restricciones del IoT industrial.

3. METODOLOGÍA

Diseño de Investigación El presente estudio se desarrolló bajo un enfoque cuantitativo, de tipo aplicado y con un nivel explicativo. Se empleó un diseño experimental comparativo para evaluar la influencia de la optimización de modelos en su eficacia predictiva y eficiencia computacional. Todo el procesamiento y experimentación se llevó a cabo en un entorno de Google Colab utilizando herramientas como Python, Scikit-learn, LightGBM, Pandas y NumPy.

Población, Muestra y Dataset La población de estudio consistió en registros de tráfico de red y eventos IoT utilizados para la detección de anomalías. La muestra extraída correspondió a un subconjunto del dataset *Edge-IIoTset* (ML-EdgeIIoT-dataset) compuesto originalmente por 157,800 muestras y 63 características. La variable objetivo (*Attack_type*) es de naturaleza multiclase y está conformada por 15 categorías, que incluyen tráfico normal (15.40%) y catorce tipos de ataques cibernéticos, tales como ataques DDoS (HTTP, ICMP, TCP, UDP), Ransomware, inyección SQL y MITM, entre otros.

Preprocesamiento de Datos Para asegurar la calidad de los datos y evitar el *data leakage*, se eliminaron las columnas no numéricas del dataset (tales como frame.time, ip.src_host, ip.dst_host, entre otras). El manejo de valores nulos se realizó mediante la imputación con la mediana de cada característica, y los valores infinitos se reemplazaron por cero, resultando en un conjunto limpio de 42 características predictivas. La variable objetivo fue transformada a formato numérico utilizando LabelEncoder. Finalmente, el dataset se dividió de manera estratificada en un conjunto de entrenamiento (80%, equivalente a 126,240 muestras) y un conjunto de prueba (20%, equivalente a 31,560 muestras), fijando una semilla aleatoria (random_state=42) para garantizar la reproducibilidad.

Modelo Base (LightGBM) Se utilizó el algoritmo Light Gradient Boosting Machine (LightGBM) como modelo base debido a su eficiencia nativa. Los hiperparámetros globales se configuraron para un problema de clasificación multiclase (objective: multiclass), con una tasa de aprendizaje (learning_rate) de 0.05, partición de características y ensacado (feature_fraction y bagging_fraction) de 0.8, límite de muestras por hoja (min_child_samples) de 20 y regularización L1/L2 de 0.1. El modelo se entrenó inicialmente con las 42 características disponibles sin aplicar ninguna optimización adicional.

Estrategias de Optimización El estudio implementó dos estrategias de optimización orientadas a reducir los requerimientos computacionales, evaluadas de manera aislada y combinada:

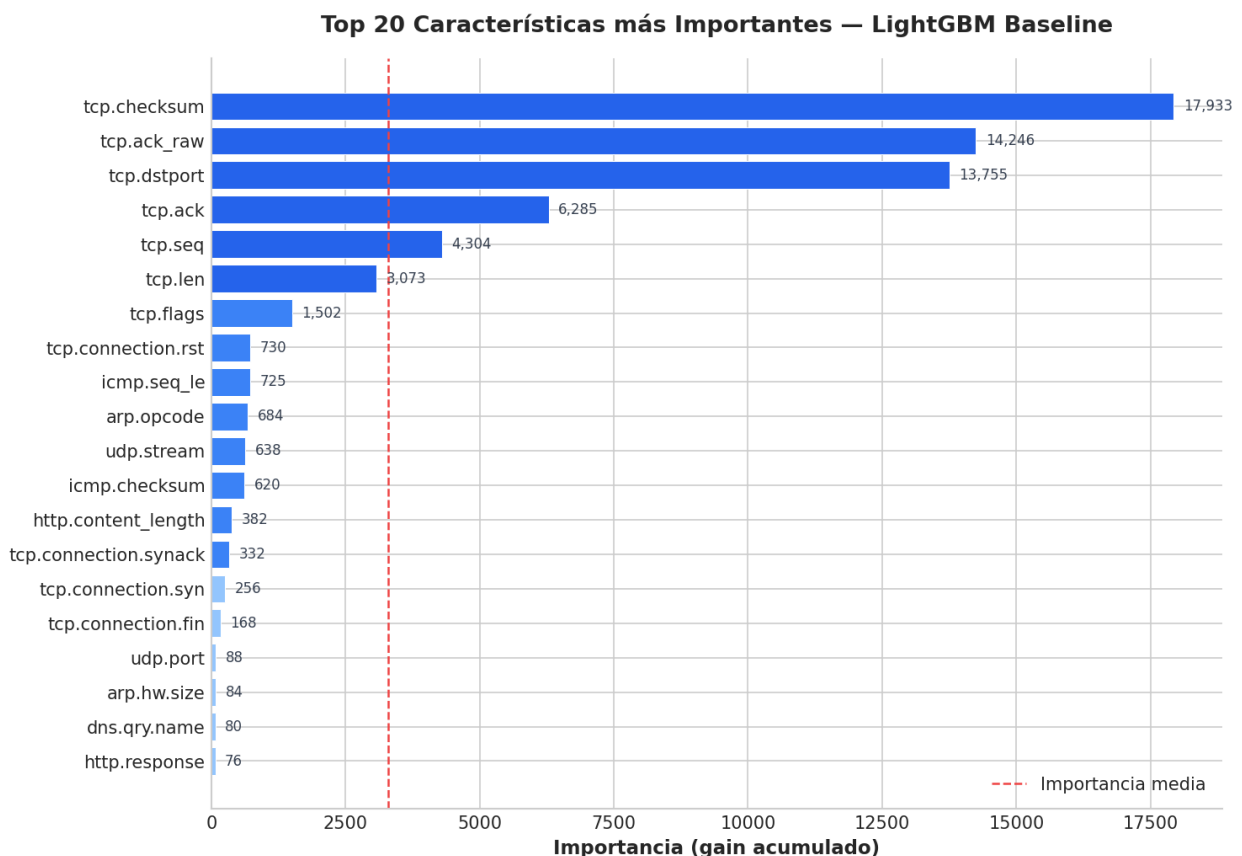
- 1. Pruning basado en la importancia de características:** Se extrajo el ranking de importancia de características (basado en la ganancia acumulada) del modelo LightGBM base. De este análisis, se identificó que atributos como tcp.checksum, tcp.ack_raw y tcp.dstport aportaban la mayor cantidad de información. Posteriormente, se retuvieron únicamente las 20 características más importantes (Top 20), logrando una reducción de dimensionalidad del 52.4%. El modelo fue reentrenado exclusivamente con este subconjunto reducido.
- 2. Compresión Ligera mediante Joblib:** Para abordar la limitación del tamaño de almacenamiento, se aplicó una serialización comprimida del modelo entrenado utilizando la librería Joblib. Se configuró un nivel de compresión alto (compress=9) para minimizar la huella de memoria del archivo .pkl resultante sin alterar la arquitectura interna ni la lógica de inferencia del modelo.

Diseño Experimental Comparativo Se estructuró un pipeline experimental para comparar cuatro escenarios de despliegue:

- 1. Modelo Base:** LightGBM estándar entrenado con las 42 características.
- 2. Pruning:** LightGBM reentrenado con las 20 características más relevantes.
- 3. Compresión Ligera:** Modelo base serializado con compresión nivel 9.
- 4. Pruning + Compresión Ligera (Co-optimización):** Modelo podado y posteriormente serializado con compresión.

Métricas Evaluadas Las métricas se dividieron en dos dimensiones fundamentales según la matriz de consistencia del estudio:

- **Eficacia Predictiva:** Para asegurar que la optimización no degradara el rendimiento de detección, se evaluaron el *Accuracy*, *Precision*, *Recall* y *F1-Score* sobre el conjunto de prueba.
- **Eficiencia Computacional y de Recursos:** Se midieron variables críticas para entornos limitados, extrayendo la Latencia de inferencia en milisegundos (promediada sobre 1000 muestras y estabilizada mediante 5 repeticiones), el Uso de memoria RAM del proceso experimental local en MB (monitorizado con la librería psutil), y el Tamaño del modelo serializado en KB (reducción de almacenamiento).



4. MODELOS UTILIZADOS

Para evaluar la viabilidad del despliegue de inteligencia artificial en entornos IoT con restricciones computacionales, se diseñó un pipeline experimental compuesto por cuatro configuraciones basadas en el algoritmo Light Gradient Boosting Machine (LightGBM). Cada configuración busca abordar distintos cuellos de botella característicos de los dispositivos de borde (edge devices), tales como la capacidad de procesamiento, la memoria disponible y el almacenamiento.

A continuación, se describen los modelos y estrategias implementadas:

Modelo Baseline (LightGBM) Se empleó LightGBM como algoritmo fundamental debido a su eficiencia inherente y rapidez en el entrenamiento mediante el uso de árboles de decisión basados en histogramas. Este modelo base fue entrenado utilizando la totalidad del espacio de características original (42 variables predictivas) y se configuró para clasificación multiclase (`objective: multiclass`) sin aplicar técnicas adicionales de optimización.

- *Propósito y relación con IoT edge:* El objetivo de esta configuración es establecer una línea de referencia estricta (benchmark). Permite cuantificar el desempeño predictivo nominal (Accuracy, Precision, Recall, F1-Score) y el consumo bruto de recursos (latencia de inferencia, uso de RAM y tamaño de almacenamiento) que un dispositivo de borde tendría que soportar si se desplegara el modelo en su estado original.

Pruning basado en selección de características por importancia A diferencia del *pruning* estructural de redes neuronales, en este enfoque basado en árboles, la poda se implementó a través de una agresiva selección de características. A partir del modelo baseline, se extrajo el ranking de importancia de los atributos (basado en la ganancia de información acumulada) y se retuvieron exclusivamente las 20 características de mayor

impacto predictivo (Top 20). Posteriormente, se entrenó un nuevo modelo restringido a este subconjunto.

- *Propósito y relación con IoT edge:* Los dispositivos IoT suelen contar con procesadores de baja potencia y memoria RAM muy limitada. Al reducir la dimensionalidad de los datos de entrada en un 52.4%, esta estrategia disminuye drásticamente la carga de procesamiento computacional requerida para atravesar las reglas de decisión, reduciendo así la latencia de inferencia de manera sustancial.

Compresión Ligera mediante Joblib Esta estrategia interviene directamente en la persistencia del modelo. Consiste en tomar el modelo baseline y aplicar una serialización comprimida utilizando la biblioteca `Joblib` configurada con un nivel de compresión alto (`compress=9`).

- *Propósito y relación con IoT edge:* En arquitecturas IoT de borde, el espacio de almacenamiento físico (memoria flash o ROM) es una limitante crítica. El propósito de la compresión ligera es minimizar la huella física del modelo (el tamaño del archivo `.pkl`). Esta técnica reduce drásticamente el peso de almacenamiento necesario para el despliegue del modelo sin afectar en absoluto su estructura interna, su lógica matemática ni su capacidad de acierto.

Combinación Pruning + Compresión Ligera (Co-optimización) Esta configuración híbrida aplica de manera secuencial las dos técnicas anteriores. Primero, el modelo es reentrenado exclusivamente con las 20 características más importantes obtenidas en el *pruning*; luego, este modelo podado es serializado utilizando la máxima compresión de `Joblib`.

- *Propósito y relación con IoT edge:* Representa la estrategia integral definitiva para despliegues reales en el borde. El propósito de esta co-optimización es

abordar simultáneamente todas las restricciones del dispositivo IoT: la poda reduce el tiempo de procesamiento (CPU) y la memoria activa (RAM) durante la inferencia, mientras que la compresión minimiza los requisitos de almacenamiento. De este modo, se logra el equilibrio óptimo, produciendo un modelo altamente veloz y liviano capaz de detectar anomalías en tiempo real sin saturar los limitados recursos del hardware.

5. RESULTADOS

Eficacia Predictiva La evaluación del desempeño predictivo reveló que las estrategias de optimización no introdujeron degradación en la capacidad de detección del modelo. El modelo baseline registró un *Accuracy* de 0.9260, un *Precision* de 0.9357, un *Recall* de 0.9260 y un *F1-Score* de 0.9282. La implementación de la compresión ligera de forma aislada conservó estas métricas exactamente en los mismos valores. Por el contrario, la aplicación de *pruning* —tanto de forma individual como combinada con compresión— generó un incremento marginal en el desempeño, alcanzando un *Accuracy* de 0.9267, *Precision* de 0.9366, *Recall* de 0.9267 y un *F1-Score* de 0.9290, lo que representa una variación del +0.09% respecto al modelo base.

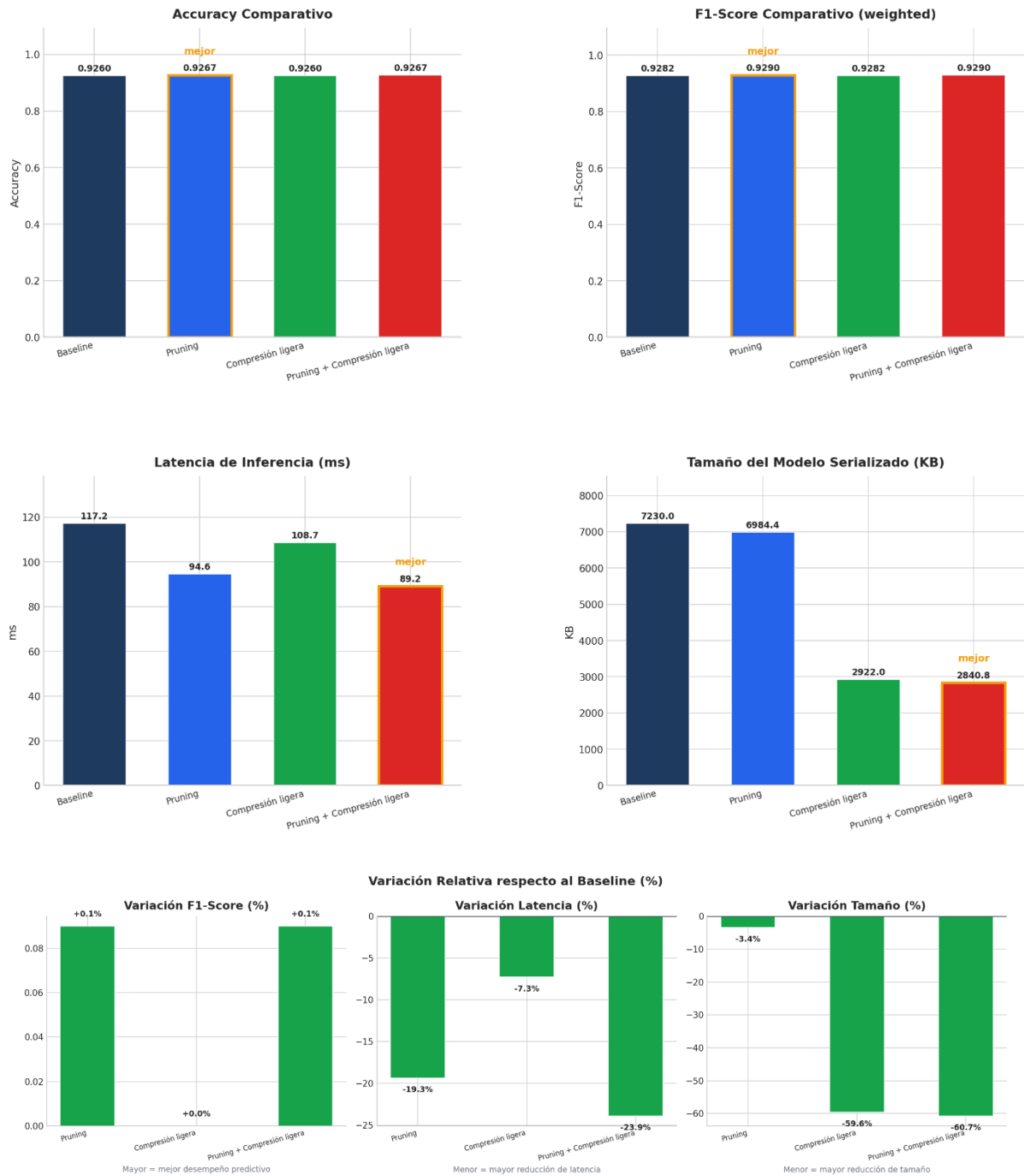
Eficiencia Computacional y de Recursos Respecto al consumo de recursos y la velocidad de ejecución, se evidenciaron mejoras sustanciales. En cuanto a la **latencia de inferencia**, el modelo baseline reportó un tiempo de 117.22 ms. La técnica de *pruning* disminuyó este valor a 94.57 ms (una mejora del 19.32%), mientras que la compresión ligera logró una ligera reducción a 108.71 ms (-7.26%). El modelo con *pruning* y compresión ligera (co-optimización) obtuvo la respuesta más rápida con 89.23 ms, marcando una reducción total de latencia del 23.88% frente al baseline.

En relación al **tamaño del modelo serializado**, el baseline presentó una huella de almacenamiento de 7230.03 KB. La aplicación exclusiva de *pruning* tuvo un impacto conservador, reduciendo el tamaño a 6984.43 KB (-3.40%). Sin embargo, la compresión ligera redujo el espacio necesario de forma notable a 2922.04 KB (-59.58%). Nuevamente, la estrategia combinada obtuvo el resultado óptimo, comprimiendo el tamaño final a 2840.76 KB, lo que equivale a una reducción del 60.71% en comparación con el modelo original.

Análisis del Trade-off El análisis de los resultados demuestra que no existe un *trade-off* negativo entre la eficacia predictiva y la eficiencia computacional bajo las técnicas empleadas. Lejos de experimentar una penalización en su capacidad de acierto, la reducción agresiva de la dimensionalidad (mediante la selección de las 20 características principales) y la compresión del archivo permitieron mejorar marginalmente las tasas de detección (+0.09% en F1-Score). En paralelo, la estrategia híbrida de *pruning* y compresión ligera mitigó significativamente los cuellos de botella característicos del despliegue en el borde, optimizando el tiempo de respuesta en casi un cuarto (-23.88%) y minimizando el espacio de almacenamiento en más de la mitad (-60.71%). Esto corrobora que la co-optimización es la configuración más apta y equilibrada para el despliegue de LightGBM en dispositivos IoT con recursos limitados.

Modelo	Accuracy	F1-Score	Latencia (ms)	Tamaño (KB)
Baseline	0.926	0.9282	117.22	7230.03
Pruning	0.9267	0.929	94.57	6984.43
Compresión Ligera	0.926	0.9282	108.71	2922.04
Pruning + Compresión Ligera	0.9267	0.929	89.23	2840.76

Dashboard Comparativo



6. DISCUSIÓN

Los resultados de este estudio demuestran que la optimización conjunta mediante la selección de características (pruning) y la compresión ligera permite reducir drásticamente los requerimientos computacionales del modelo LightGBM sin comprometer su capacidad

predictiva. La estrategia combinada logró una reducción del 60.71% en el tamaño de almacenamiento y del 23.88% en la latencia de inferencia, mejorando de forma marginal la tasa de detección (+0.09% en F1-Score).

Relación con investigaciones previas y comparación de resultados Estos hallazgos son altamente consistentes con la literatura reciente enfocada en optimización de aprendizaje automático para ecosistemas IoT de borde. Investigaciones previas como la de Hadi et al. evaluaron la optimización de LightGBM para la detección de anomalías eléctricas en hardware IoT, demostrando que el *pruning* puede preservar la exactitud predictiva y producir matrices de confusión idénticas al modelo original, al tiempo que reduce considerablemente el peso del archivo. De manera análoga, el presente trabajo comprobó que podar el 52.4% de las características de entrada no penalizó el rendimiento, sino que estabilizó la clasificación.

Asimismo, la importancia de reducir la dimensionalidad ha sido respaldada por Khan et al., quienes demostraron que técnicas de selección de características (en su caso, mediante algoritmos genéticos) son fundamentales para eliminar redundancias, acelerar la detección y reducir el tamaño del modelo. Aunque nuestro modelo alcanzó un tamaño final de 2840.76 KB tras la compresión, el fundamento es el mismo: retener únicamente el subconjunto de variables (Top 20 por ganancia de información) que maximiza el rendimiento bajo restricciones operativas.

Ventajas del pruning y la compresión ligera La ventaja fundamental del pruning basado en características en modelos de árboles es que disminuye de forma directa la cantidad de reglas de decisión y evaluaciones lógicas que el procesador debe realizar durante la fase de inferencia. Esto se traduce en una menor carga computacional y un descenso efectivo en la latencia de predicción. Por su parte, la compresión ligera mediante Joblib es vital para solucionar las restricciones de almacenamiento físico (memoria ROM/Flash) típicas de los nodos IoT. Al basarse en algoritmos de serialización comprimida, esta técnica reduce

la huella del modelo en disco de manera segura y no destructiva, evitando las complejas calibraciones matemáticas requeridas por técnicas de compresión estructural como la cuantización.

Limitaciones del entorno experimental en Google Colab Es imperativo reconocer que este experimento se desarrolló y simuló completamente en un entorno virtualizado en la nube (Google Colab) mediante librerías de monitoreo como `psutil` y `time`. Esto conlleva limitaciones significativas respecto al comportamiento real del hardware:

- La latencia de inferencia y el consumo de RAM medidos en los procesos del notebook no reflejan con total fidelidad el comportamiento de microcontroladores o procesadores ARM (como una Raspberry Pi) bajo condiciones de sistemas operativos de tiempo real.
- El consumo energético y el estrangulamiento térmico, métricas críticas para la viabilidad de dispositivos IoT alimentados por baterías que otros autores han perfilado mediante herramientas físicas como medidores de potencia USB e instrumentación de temperatura, no pudieron ser medidos experimentalmente en este entorno virtual.

Futuras líneas de investigación Para extender y validar la viabilidad práctica de los hallazgos de este estudio, las investigaciones futuras deben enfocarse en el despliegue físico en hardware de borde (edge devices) real (ej. Raspberry Pi 4/5 o Microcontroladores con TensorFlow Lite) para medir la latencia física y el gasto energético empírico. Además, se sugiere comparar el desempeño del modelo actual contra optimizaciones más agresivas, como la cuantización a enteros de 8 bits (INT8) y el uso de compiladores especializados de inferencia para árboles de decisión (como Treelite o rutinas ONNX), los cuales han probado generar aceleraciones muy superiores para arquitecturas LightGBM en el borde. Finalmente, evaluar este modelo ligero bajo arquitecturas de aprendizaje federado (Federated Learning)

permitiría escalar la detección de intrusiones en infraestructuras industriales de forma colaborativa y preservando la privacidad.

7. CONCLUSIONES

El presente estudio cumplió con su objetivo principal al evaluar la influencia de la optimización de modelos de inteligencia artificial para entornos IoT. Los resultados experimentales validan categóricamente la hipótesis planteada: la implementación conjunta de *pruning* (mediante selección de características) y compresión ligera mejora significativamente la ejecución eficiente de modelos de IA en dispositivos de borde con recursos limitados.

Los hallazgos principales demuestran que es posible resolver los cuellos de botella de memoria y procesamiento sin degradar la capacidad de detección. La estrategia de co-optimización aplicada al modelo LightGBM logró una **reducción del 60.71% en el tamaño de almacenamiento** (pasando de 7230.03 KB a 2840.76 KB) y una **disminución del 23.88% en la latencia de inferencia** (de 117.22 ms a 89.23 ms). Además, esta agresiva reducción estructural estabilizó la clasificación, incrementando marginalmente la eficacia predictiva con una mejora del +0.09% en el *F1-Score*.

En cuanto a las limitaciones, cabe destacar que la experimentación se llevó a cabo en un entorno virtualizado (Google Colab). En consecuencia, las métricas de latencia y memoria representan aproximaciones algorítmicas que no capturan fenómenos inherentes al hardware físico real, tales como el estrangulamiento térmico (*thermal throttling*) bajo carga sostenida o el consumo energético en dispositivos IoT alimentados por baterías.

Para trabajos futuros, se propone el despliegue empírico del modelo en hardware de borde físico (como una Raspberry Pi o microcontroladores dedicados) para auditar el gasto energético y la latencia real. Adicionalmente, se sugiere investigar la combinación de este

modelo ligero con técnicas de **cuantización a enteros de 8 bits (INT8)**, el uso de compiladores especializados (como Treelite u ONNX) para acelerar la inferencia en arquitecturas ARM, y la evaluación del sistema bajo esquemas de **Aprendizaje Federado (Federated Learning)** para garantizar la privacidad de los datos en aplicaciones críticas industriales.

REFERENCIAS

1. Fridous, V. M., Agarwal, A., Bhaskar, K. B., Nandagopal, V., & Sivakamasundari, N. (2026). Optimization and Benchmarking of Lightweight Neural Networks for Efficient Embedded AI Deployment. *Engineering Reports*, 8(5). <https://doi.org/10.1002/eng2.70814>
2. Miranto, A., Prasetyawan, P., & Aryanto, I. M. (2026). Optimization of MobileNet SSD Using Pruning, Quantization, and Transfer Learning for Real-Time Vehicle Detection in IoT-Based Security Systems. *Jurnal Teknik Informatika (Jutif)*, 7(2), 1500-1514. <https://doi.org/10.52436/1.jutif.2026.7.2.5488>
3. Hadi, S., Syafei, W. A., Wibowo, A., Hassanudin, W. M., Setiana, E. F., & Putri, A. N. (2026). A Comparative Analysis of Pruning, Quantization, and Compilation for LightGBM-Based Electricity Anomaly Detection on IoT Edge Devices. *Engineering Technology & Applied Science Research*, 16(2), 32935-32941. <https://doi.org/10.48084/etasr.16433>
4. Khowaja, S. A., Waqas, A., Quasim, M. T., & Dev, K. (2025). POST: Pruning Oriented Security for Inversion Attack in Edge-Based Internet of Things. *IEEE Internet Of Things Journal*, 13(3), 4411-4420. <https://doi.org/10.1109/jiot.2025.3632564>

5. Khan, A., Hussain, M. A., & Anwer, F. (2025). A Hybrid Lightweight Deep Learning-Based Intrusion Detection Approach in IoT Utilizing Feature Selection & Explainable Artificial Intelligence. *IEEE Access*, 13, 192451-192466. <https://doi.org/10.1109/access.2025.3630753>
6. Akhi, M., Eising, C., & Dhirani, L. L. (2025). Securing IoT Using Lightweight TCN for Edge Deployment on Raspberry Pi 4. *IEEE Open Journal Of The Communications Society*, 7, 442-460. <https://doi.org/10.1109/ojcoms.2025.3649498>
7. Adjewa, F., Esseghir, M., & Merghem-Boulahia, L. (2026). From Edge Transformer to IoT Decisions: Offloaded Embeddings for Lightweight Intrusion Detection. *Sensors*, 26(2), 356. <https://doi.org/10.3390/s26020356>
8. Wang, Z., Li, Z., He, D., & Chan, S. (2022). A lightweight approach for network intrusion detection in industrial cyber-physical systems based on knowledge distillation and deep metric learning. *Expert Systems With Applications*, 206, 117671. <https://doi.org/10.1016/j.eswa.2022.117671>
9. Roy, S., Li, J., Choi, B., & Bai, Y. (2021). A lightweight supervised intrusion detection mechanism for IoT networks. *Future Generation Computer Systems*, 127, 276-285. <https://doi.org/10.1016/j.future.2021.09.027>
10. Jan, S. U., Ahmed, S., Shakhov, V., & Koo, I. (2019). Toward a Lightweight Intrusion Detection System for the Internet of Things. *IEEE Access*, 7, 42450-42471. <https://doi.org/10.1109/access.2019.2907965>